# Chapter 12. Data Science and R

## 12.0 Introduction

Data science is a relatively new discipline that first came to the attention of many with a 2010 article by O'Reilly's Mike Loukides. While there are many definitions in the field, Loukides distills his detailed observation of and participation in data science into this definition:

> *A data application acquires its value from the data itself, and creates more data as a result. It's not just an application with data; it's a data product. Data science enables the creation of data products.*

One of the main open source ecosystems for data science software is at Apache and includes Hadoop (which includes the Hadoop Distributed File System [HDFS], Hadoop MapReduce,[1] the Ozone object store, and the YARN scheduler), the Cassandra distributed database, and the Spark compute engine. Read the Modules and Related projects sections of the Hadoop page for a current list.

What's interesting here is that a great deal of this infrastructure, which is taken for granted by data scientists, is written in Java and Scala (a JVM language). Much of the rest is written in Python, a language that complements Java. Many users see only the Python side of things and don't realize that Java is behind some of the infrastructure.

Data science (DS) problems can involve a lot of setup, so we'll give only one example from traditional DS, using the Spark framework. Spark is written in Scala, so it can be used directly by Java code.

In the rest of the chapter I'll focus on a language called R, which is widely used both in statistics and in data science (and many other sciences; many of the graphs you see in refereed journal articles are prepared with R). Because R is so widely used it is useful to know. Its primary implementation was not written in Java, but in a mixture of C, Fortran, and R itself. But R can be used within Java, and Java can be used within R. I'll talk about

several implementations of R and how to select one, and then I'll show techniques for using Java from R and R from Java, as well as using R in a web application.

Some Java-centric alternatives to R are DataFrame libraries such as dflib (*https://dflib.org*) and dataframe-ec (*https://github.com/vmzakharov/data-frame-ec*). The related topic of machine learning for artificial intelligence is treated separately in Chapter 13.

---

TIP

Nothing in this chapter is required to understand the later chapters; it is here as a discussion of one significant area of Java's applicability.

---

# 12.1 Using Data in Apache Spark

## Problem

You want to process data using Spark.

## Solution

Create a `SparkSession`, use its `read()` function to read a `DataSet`, apply operations, and summarize results.

## Discussion

Spark supports data science operations on one or more computers. The Apache Spark website reads, in part:

> *Apache Spark™ is a multi-language engine for executing data engineering, data science, and machine learning on single-node machines or clusters...[Spark is] the most widely-used engine for scalable computing.*

Quoting Databricks, home of much of the original Spark team (viewed in 2022):[2]

*Apache Spark™ has seen immense growth over the past several years, becoming the de-facto data processing and AI engine in enterprises today due to its speed, ease of use, and sophisticated analytics. Spark unifies data and AI by simplifying data preparation at massive scale across various sources, providing a consistent set of APIs for both data engineering and data science workloads, as well as seamless integration with popular AI frameworks and libraries such as TensorFlow, PyTorch, R and SciKit-Learn.*

One thing Spark is good for is dealing with lots of data. Example 12-1 reads an Apache-format logfile and finds (and counts) the lines with 200, 404, and 500 responses. The logfile consists of lines like this, with the status code in a later column. We use a regex (see Chapter 4) to find the status code as a word by itself, preceded and followed by at least one nondigit (the `\D` in the regex).

**Example 12-1. spark/src/main/java/sparkdemo/LogReader.java**

```
/**
 * Read an Apache logfile and summarize it.
 */
public class LogReader {

  public static void main(String[] args) {

    final String logFile = "/var/wildfly/standalone/log/access_log.log"; ❶
    SparkSession spark =
      SparkSession
            .builder()
            .appName("Log Analyzer")
            .config("spark.master", "local")
            .getOrCreate();                                    ❷
    Dataset<String> logData = spark.read().textFile(logFile).cache();   ❸

    long good = logData.filter(                                ❹
    new FilterFunction<>() {public boolean call(String s) {
        return s.contains(".*\\D200\\D.*");
      }
     }).count();

    long bad = logData.filter(new FilterFunction<>() {
      public boolean call(String s) {
        return s.matches(".*\\D404\\D.*");
      }
     }).count();
```

```
    long ugly = logData.filter(new FilterFunction<>() {
      public boolean call(String s) {
       return s.matches(".*\\D500\\D.*");
      }
     }).count();

    System.out.printf(                                    ❺
     "Successful transfers %d, 404 tries %d, 500 errors %d\n",
     good, bad, ugly);

    spark.stop();
  }
}
```

❶ Set up the filename for the logfile. It probably should come from `args`.

❷ Start up the `SparkSession` object—the runtime.

❸ Tell Spark to read the logfile and keep it in memory (cache).

❹ Define the filters for 200, 404, and 500 errors. They should be able to use lambdas to make the code shorter, but there's an ambiguity between the Java and Scala versions of `FilterFunction`.

❺ Print the results.

We can't run this as a regular Java program, as it depends on the Spark runtime. To make this compile, you need to add the following to a Maven POM file:

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.12</artifactId>
  <version>2.4.4</version>
  <scope>provided</scope>
</dependency>
```

Then you should be able to use `mvn package` to get a JAR file packaged.

To run the app, note the use of the `provided` scope, because we will also download the Apache Spark runtime package from the Spark Download page. Unpack the distribution and set the `SPARK_HOME` environment to its root, i.e., the directory created when unpacking the download (you may find a newer version when you run this):

```
SPARK_HOME=~/spark-3.5.1-bin-hadoop3/
```

Then you can use the `run` script that I've provided in the source download (*javasrc/spark*).

Spark is designed for larger-scale computing than what's in this simple example, so its voluminous output dwarfs the output from my basic sample program. Nonetheless, for an approximately 42,000-line file, I did get this result, buried among the logging:

```
Successful transfers 32555, 404 failures 6539, 500 errors 183
```

As mentioned in "Standard input and output streams", there are two output streams from a normal program: the standard output (stdout) and the standard error (stderr). Fortunately, Spark puts all the debug chatter on stderr, so we can redirect the stdout to a file, like this:

```
run 2>spark.log
```

Then only the one line of useful output will be printed.

Finally, if Docker is supported and installed on your system, you can get an interactive Spark session by using:

```
docker run -it --rm spark /opt/spark/bin/spark-shell
```

## See Also

Spark is a massive subject, but a necessary tool for many data scientists. You can program Spark in Java (obviously), Scala (a JVM language that promotes functional programming), Python, and probably other languages. You can learn more at the Apache Spark website or from the books, videos, and tutorials online.

# 12.2 Using R Interactively

## Problem

You don't know the first thing about R, but you want to.

## Solution

R has been around for ages, with its predecessor, S, existing for a decade before that. There are many books and online resources devoted to this language, as well as an official home page. There are many online tutorials; the R Project hosts one. R itself is available in most systems' package managers, and it can be downloaded from the official download site. The name *CRAN* in these URLs stands for Comprehensive R Archive Network, named in a similar fashion to TeX's CTAN and the Perl language's CPAN.

In this example we'll write some data from a Java program and then analyze and graph it using R interactively.

## Discussion

This is merely a brief intro to using R interactively. Suffice to say that R is a valuable interactive environment for exploring data. Here are some simple calculations to show the flavor of the language: a chatty startup (so long I had to cut part of it), simple arithmetic, automatic printing of results if not saved, half-decent errors when you make a mistake, and arithmetic on vectors. You might see some similarities to Java's JShell (see Recipe 1.5); both are REPL (read-evaluate-print loop) interfaces. R adds the ability to save your interactive session (called a *workspace*) when exiting the program, so all your data and function definitions are restored next time you start R. A simple interactive session showing a bit of R's syntax might look like this:

```
$ R

R version 4.2.3 (2023-03-15) -- "Shortstop Beagle"
Copyright (C) 2023 The R Foundation for Statistical Computing
Platform: x86_64-unknown-openbsd7.5 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
```

```
...

> 2 + 2
[1] 4
> x = 2 + 2
> x
[1] 4
> r = 10 20 30 40 50
Error: unexpected numeric constant in "r = 10 20"
> r = c(10,20,30,45,55,67)
> r
[1] 10 20 30 45 55 67
> r+3
[1] 13 23 33 48 58 70
> r / 3
[1]  3.333333  6.666667 10.000000 15.000000 18.333333 22.333333
>quit()
Save workspace image? [y/n/c]: n
$
```

R purists will usually use the *assignment arrow* `<-` in lieu of the `=` sign when assigning. If you like that, go for it.

This short session barely scratches the surface: R offers hundreds of built-in functions, sample datasets, over a thousand add-on packages, built-in help, and much more. For interactive exploration of data, R really is the one to beat.

Some people prefer a GUI frontend to R. There are several:

- R Studio is a GUI for the standard implementation of R.
- Ride is a GUI frontend for Renjin, an alternative implementation of R written in Java.

Now we want to write some data from Java and process it in R (we'll use Java and R together in later recipes in this chapter). In Recipe 5.9 we discussed the `java.util.Random` class and its `nextDouble()` and `nextGaussian()` methods. The `nextDouble()` and related methods try to give a flat distribution between 0 and 1.0, in which each value has an equal chance of being selected. A Gaussian or normal distribution is a bell curve of values from negative infinity to positive infinity, with the majority of the values clustered around zero (0.0). We'll use R's histogramming and graphics functions to examine visually how well they do this:

```
Random r = new Random();
for (int i = 0; i < 10_000; i++) {
   System.out.println("A normal random double is " + r.nextDouble());
   System.out.println("A gaussian random double is " + r.nextGaussian());
```

To illustrate the different distributions, I generated 10,000 numbers each using `nextRandom()` and `nextGaussian()`. The code for this is in *Random4.java* (not shown here) and is a combination of the preceding sample code with code to print just the numbers into two files. I then plotted histograms using R; the R script used to generate the graph is in *javasrc* under *src/main/resources*, but its core is shown in Example 12-2. The `read.table()` function reads a text file into tabular form. The `layout()` says we want two graphs side by side, and the `hist()` call generates a histogram, naturally.

**Example 12-2. R commands to generate histograms**

```
png("randomness.png")
us <- read.table("normal.txt")[[1]]
ns <- read.table("gaussian.txt")[[1]]

layout(t(c(1,2)), respect=TRUE)

hist(us, main = "Using nextRandom()", nclass = 10,
     xlab = NULL, col = "lightgray", las = 1, font.lab = 3)

hist(ns, main = "Using nextGaussian()", nclass = 16,
     xlab = NULL, col = "lightgray", las = 1, font.lab = 3)
dev.off()
```

The `png()` call tells R which graphics device to use. Others include `X11()` and `Postscript()`. `read.table()` reads data from a text file into a table; the `[1]` gives us just the data column, ignoring the metadata. The `layout()` call says we want two graphics objects displayed side by side. Each `hist()` call draws one of the two histograms. And `dev.off()` closes the output and flushes any writing buffers to the PNG file. The result is shown in Figure 12-1.

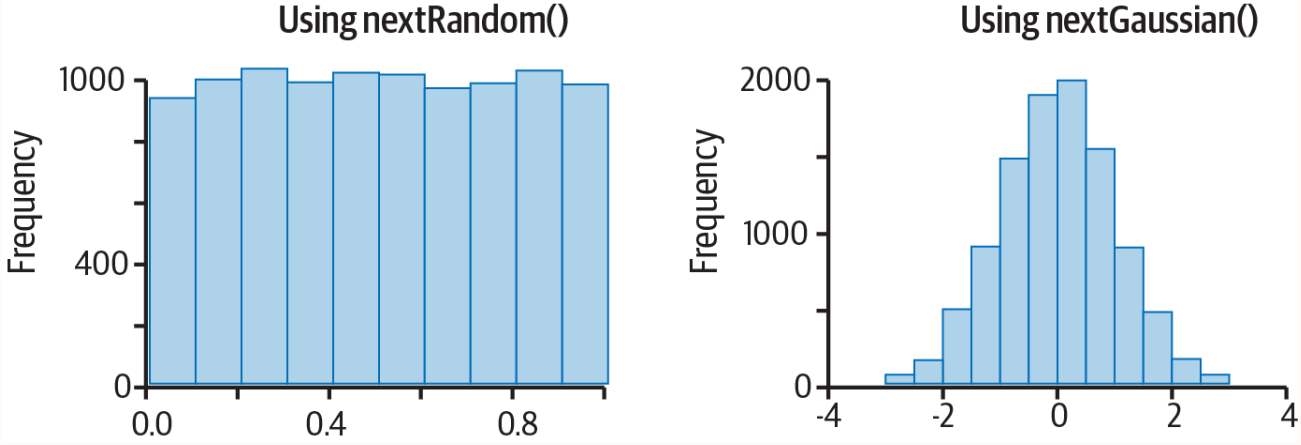



Figure 12-1. Flat (left) and Gaussian (right) distributions

# 12.3 Comparing/Choosing an R Implementation

## Problem

You're not sure which implementation of R to use.

## Solution

Look at original R, Renjin, and FastR.

## Discussion

Before there was R, there was S, an environment for interactive statistics and programming developed by John Chambers and others at AT&T Bell Labs starting in 1976. I ran into S when supporting the University of Toronto Statistics Department, and again when reviewing a commercial implementation of S, called S-PLUS, for a long-ago glossy magazine called *Sun Expert*. AT&T was only making S source code available to universities and commercial licensees who could not further distribute the code. Frustrated by this policy, two developers at the University of Auckland, Ross Ihaka and Robert Gentleman, developed a clone of S in 1995. They named it R after their own first initials and as a play on the name S. (There is precedent for this: the AWK language popular on Unix/Linux was named for the initials of its designers, Aho, Weinberger, and Kernighan). R grew quickly because it was very compatible with S and was more readily available. The original implementation of R is actively managed by the R Foundation for Statistical Computing, which also manages the Comprehensive R Archive Network.

Renjin is a fairly complete implementation of R in Java. This project provides built JAR files via their own Maven repository.

FastR is another implementation in Java, designed to run as fast as the original R in the faster Graal VM, that supports direct invocation of JVM code from almost any other programming language. Originally a university project, FastR was shepherded by Oracle as part of `Graal VM`, but is no longer actively maintained by them.

Besides these implementations, R's popularity has led to the development of access libraries for invoking R from many popular programming languages. Rserve is a TCP/IP networked access mode for R, for which Java wrappers exist.

The rest of this chapter covers how to use R from Java, and vice versa.

# 12.4 Using R from Within a Java App: Renjin

## Problem

You want to use Renjin to access R from within a Java application.

## Solution

Add Renjin to your Maven or Gradle build, and call it via the Scripting Engine mechanism described in Recipe 18.3.

## Discussion

Renjin is a pure-Java, open source reimplementation of R and provides a `ScriptEngine` interface (see Recipe 18.3). Add the following dependency to your build tool:

```
org.renjin:renjin-script-engine:3.5-beta76
```

Of course, by the time you read this, there is probably a later version of Renjin available than the one shown here; use the latest version unless there's a reason not to.

Note that you will also need a `<repository>` entry since the maintainers put their artifacts in the repo at *nexus.betadriven.com* instead of the usual Maven Central. Here's what I used (obtained from the Renjin downloads page):

```
<repositories>
  <repository>
```

```
        <id>bedatadriven</id>
        <name>bedatadriven public repo</name>
        <url>https://nexus.bedatadriven.com/content/groups/public/</url>
    </repository>
</repositories>
```

Once that's done, you should be able to access Renjin via the Scripting Engine framework, as in Example 12-3. This runs nicely under an IDE that provides the `CLASSPATH`. To run it under command-line Maven you'd use:

```
mvn exec:java -Dexec.mainClass="otherlang.RenjinScripting"
```

If the application will be used often you could package it, for example with `jpackage` (see Recipe 2.18).

**Example 12-3. main/src/main/java/otherlang/RenjinScripting.java**

```
/**
 * Demonstrate interacting with the "R" implementation called "Renjin"
 */
public static void main(String[] args) throws ScriptException {
  ScriptEngineManager manager = new ScriptEngineManager();
  ScriptEngine engine = manager.getEngineByName("Renjin");
  engine.put("a", 42);
  Object ret = engine.eval("b <- 2; a*b");
  System.out.println(ret);
}
```

Because R treats all numbers as floating point, like many interpreters, the value printed is `84.0`.

You can also get Renjin to invoke a script file; Example 12-4 invokes the same script used in Recipe 12.2 to generate and plot a batch of pseudorandom numbers.

**Example 12-4. main/src/main/java/numbers/Random5.java (Renjin with a script file)**

```
private static final String R_SCRIPT_FILE = "/randomnesshistograms.r";
private static final String PLOT_FILE = "randomness.png";
private static int N = 100_000;

public static void main(String[] argv) throws Exception {
```

```
    // java.util.Random methods are non-static; we need an instance
    if (argv.length == 1) {
      try {
        N = Integer.parseInt(argv[0]);
      } catch (NumberFormatException ex) {
        System.out.printf("Number %s invalid, using %d\n", argv[0], N);
      }
    }
    System.out.println("Generating " + N + " randoms");
    Random r = new Random();
    double[] us = new double[N], ns = new double[N];
    for (int i=0; i<N; i++) {
      us[i] = r.nextDouble();
      ns[i] =r.nextGaussian();
    }
    System.out.println("Generating histograms");
    try (InputStream is =
      Random5.class.getResourceAsStream(R_SCRIPT_FILE)) {
      if (is == null) {
        throw new IllegalStateException("Can't open R file ");
      }
      ScriptEngineManager manager = new ScriptEngineManager();
      ScriptEngine engine = manager.getEngineByName("Renjin");
      engine.put("us", us);
      engine.put("ns", ns);
      engine.eval(FileIO.readerToString(new InputStreamReader(is)));
    }
    System.out.println("Done; output in " + PLOT_FILE);
  }
```

Renjin can also be used as a standalone R implementation if you download an all-dependencies JAR file from the Renjin downloads page.

# 12.5 Using Java from Within an R Session

## Problem

You are partway through a computation in standard R and realize that there's a Java library available to do the next step. Or you need to call Java code from within an R session for any reason.

## Solution

Install the library rJava, call `.jinit()`, and use `J()` to load classes or invoke methods.

## Discussion

Here is the part of an interactive R session in which we install rJava, load its library into R, and invoke `java.time.LocalDate.now()` to get the current date:

```
> install.packages('rJava')                    ❶
trying URL 'https://.../rJava_0.9-11.tgz'
Content type 'application/x-gzip' length 745354 bytes (727 KB)
downloaded 727 KB

The downloaded binary packages are in
   /tmp//Rtmp6XYZ9t/downloaded_packages
> library('rJava')                             ❷
> J('java.time.LocalDate', 'now')              ❸
[1] "Java-Object{2025-11-22}"
> d=J('java.time.LocalDate', 'now')$toString()  ❹
> d
[1] "2025-11-22"
```

❶ Install the rJava package; this only needs to be done once. It produces a lot of chatter and may present warnings about missing libraries. Try ignoring those and going on to Step 2.

❷ Load the rJava library, which is needed in every R session.

❸ The `J` function takes one argument of a full class name. If only that argument is given, a class descriptor (like a `java.lang.Class` object) is returned. If more than one argument is given, the second is a static method name, and any subsequent arguments are passed to that method.

❹ Returned objects can have Java methods invoked with the standard R `$` notation; here the `toString()` method is invoked to return just a character string instead of a `LocalDate` object.

The `.jcall` function gives you more control over calling method and return types:

```
> d=J('java.time.LocalDate', 'now')                    ❶
> .jcall(d, "I", 'getYear')                            ❷
[1] 2025
>
> .jcall("java/lang/System","S","getProperty","user.dir") ❸
[1] "/home/ian"
> c=J('java/lang/System')                              ❹
> .jcall(c, "S", 'getProperty', 'user.dir')
[1] "/home/ian"
>
```

❶ Invoke the Java `LocalDate.now()` method and save the result in R variable `d`.

❷ Invoke the Java `getYear()` method on the `LocalDate` object; the `I` tells `.jcall` to expect an integer result.

❸ Call `System.getProperty("user.dir")` and print the result; the `S` tells `.jcall` to expect a string return.

❹ If you will be using a class several times, save the `Class` object and pass it as the first argument of `.jcall()`.

To invoke a constructor, call `.jnew()` with the class name and any arguments:

```
> .jnew('java.math.BigDecimal', 22.2)$doubleValue()
[1] 22.2
>
```

rJava is capable of much more; consult the documentation.

# 12.6 Using R in a Web App

## Problem

You want to display R's data and graphics in a web page on a web server.

## Solution

There are several approaches that would achieve this effect:

- Prepare the data, generate graphics as we did in Recipe 12.2, and then incorporate both into a static web page.
- Use one of several R add-on web frameworks, such as shiny or Rook.
- Invoke a JVM implementation of R from within a Servlet, JavaServer Faces (JSF), Spring bean, or other web-tier component.

## Discussion

The first approach is trivial and doesn't need discussion here.

For the second, I'll use the R package timevis, which in turn uses shiny. This isn't built into the R library, so we first have to install it, using R's `install.packages()`:

```
$ R
> install.packages('timevis')
> quit()
$
```

This may take a while as it downloads and builds multiple dependencies.

For this demo I have a small dataset with some basic information on medieval literature, which I load and display using shiny and timevis:

```
# Draw the timeline for the epics.

epics = read.table("epics.txt", header=TRUE, fill=TRUE)

# epics

library("timevis")

timevis(epics)
```

When run, this creates a temporary file containing HTML and JavaScript to allow interactive exploration of the data. The library also opens this in a browser, as shown in Figure 12-2. You can explore the data by expanding or contracting the timeline and scrolling sideways.

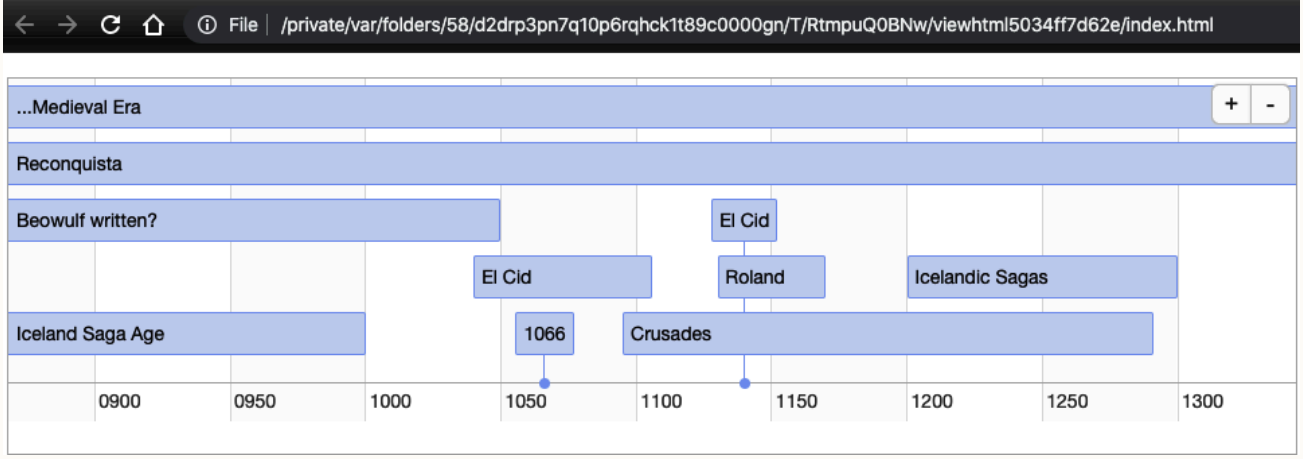



Figure 12-2. Timevis (shiny) in action

Where there are two boxes (El Cid, Icelandic Sagas), the first is when the life or stories took place, and the second is when they were written down.

To expose this on the public web, copy the file (whose full path is displayed in the browser title bar) and the *lib* folder in that same directory into a directory served by the web server. Or just use File→Save As→Complete Web Page within the browser. Either way, you must do this while the R session is running, as the temporary files are deleted when the session ends. Or, if you are familiar with the shiny framework, you can insert the timevis visualization into a shiny application.

1 *Map/Reduce* is a famous producer-consumer algorithm pioneered by Google to handle large data problems. An unspecified number of producers process *map* data—such as from words on a web page to the page's URL—and a single (or a few) reduce process reduces the maps to a manageable form, such as a list of all the pages that contain the given words. Early on, some data scientists went overboard trying to do everything with Map/Reduce; now the pendulum has swung back to using compute engines like Spark.

2 Databricks offers several free ebooks on Spark from its website; it also offers commercial Spark add-ons.

Previous chapter
< 11. Threaded Java

Next chapter
13. Machine Learning/Artificial Intelligence >